

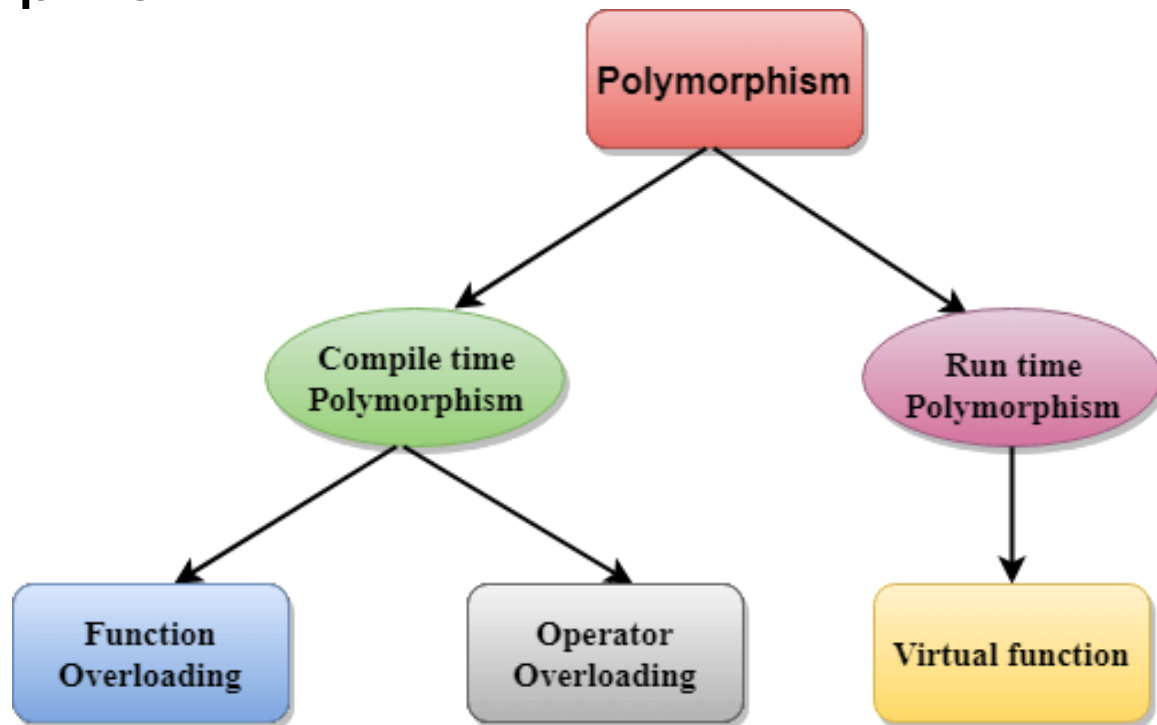
Polymorphism

Polymorphism

- The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms. It is a greek word.
- In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.
- A real-life example of polymorphism, a person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism.

Types of Polymorphism

- Compile Time Polymorphism
 - Function Overloading
 - Operator Overloading
- Runtime Polymorphism
 - Virtual Function



Compile time Polymorphism

- The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding.

Run time polymorphism

- Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.
- Runtime polymorphism is achieved through function overriding (virtual function). The function to be called/invoked is established during runtime.

Difference between Runtime and Compile time Polymorphism

Compile time polymorphism (Early Binding)	Run time polymorphism (Late Binding)
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at the compile time.	It is more flexible as all the things execute at the run time.

Pointer

- A pointer is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable , you must declare a pointer before using it to store any variable address.
- Syntax:

type *var-name;

here type can be any type like int, float or any other class type.

Example: int X=5;

int *PX;

PX=&X;

cout<<*PX;

Pointer to Object

- Object can also have an address, so there is also a pointer that can point to the address of an object. An object of a class behaves identically as any other variable.

- Example:

```
class Employee {  
    int code;  
    char name [20] ;  
    public:  
    void getdata ()0 ;  
    void display () ;  
};
```

The following codes is written

```
Employee *pEmp;
```

This declaration creates a pointer variable **pEmp** that can point to any object of employee type.

```
Employee Emp;
```

```
pEmp=&Emp;
```

Pointer to object

- Creating object using new keyword:
Employee *pEmp=new Employee;
//this statement for single object
Employee *pEmp=new Employee[10];
//for array of objects
- Accessing Class Member using pointer object:
pEmp->getdata();
pEmp[i]->getdat(); //(pEmp+i)->getdata()

this Pointer

- Every object in C++ has access to its own address through an important pointer called this pointer. The this pointer is an implicit parameter to all member functions. Therefore, inside a member function, this may be used to refer to the invoking object.
- Friend functions do not have a this pointer, because friends are not members of a class.
- Only member functions have a this pointer.

Example

```
# include<iostream>
Using namespace std;
class item {
    int number;
    float cost;
    public:
    void getdata ( int , float);
    void putdata ( void)
    {
        cout<<"Number: " <<number<<endl;
        cout<<"Cost : " <<cost<<endl;
    }
};

void item :: getdata (int number , float
cost)
{
    this->number=number;
    this->cost=cost;
}

int main ( )
{
    item x;
    cout<<"\n object x:"<<endl;
    x. getdata( 100,299.95);
    x .putdata();
    item y;
    cout<<"\n object  y"<<endl;
    y. getdata(200,175.5);
    y. putdata();
    return 0;
}
```

Pointer to Derived Class

- Pointers can be declared to point base or derived classes. Pointers to object of base class are type compatible with pointers to object of derived class.
- A base class pointer can also point to objects of base and derived classes.
- In other words, a pointer to base class object can point to objects of derived classes whereas a pointer to derived class object cannot point to objects of base class object.

For Example:

```
class base
{
//Data Members
//Member Functions
};
class derived : public base
{
//Data Members
//Member functions
};
int main ( )
{
base *ptr; //pointer to class base
derived obj ;
ptr = &obj ;
}
```

- The pointer ptr points to an object of the derived class obj. But, a pointer to a derived class object may not point to a base class object.

- the following assignment statements are not valid

```
int main ( )
{
    base obja;
    derived *ptr;
    ptr = &obja; //invalid....
}
```

Example:

```
#include <iostream>
using namespace std;
class Base
{
int x;
public:
void setx(int i) {
x = i;
}
void getx() {
cout<< x;
}
};
class Derived: public Base
{
int y;
public:
void sety(int i) {
y = i;
}
void gety() {
```

```
cout <<y;
}
};
int main()
{
Base *bptr;
Base bobj;
Derived dobj;
bptr = &bobj;
bptr->setx(10);
cout << "\nBase object x: ";
bptr->getx();
bptr = &dobj;
bptr->setx(99);
dobj.sety(88);
cout << "\nDerived object x: " ;
bptr->getx();
cout << "\nDerived object y: ";
dobj.gety();
return 0;
}
```

Virtual Function

- If there are member functions with same name in base class and derived class, virtual functions gives programmer capability to call member function of different class by a same function call depending upon different context. This feature in C++ programming is known as polymorphism which is one of the important feature of OOP.
- If a base class and derived class has same function and if you write code to access that function using pointer of base class then, the function in the base class is executed even if, the object of derived class is referenced with that pointer variable. This issue can only be resolved by using the 'virtual' function.
- In order to make a function virtual, you have to add keyword virtual in front of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Rules of Virtual Function

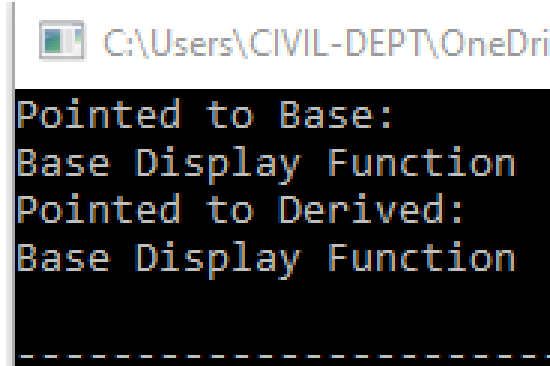
- Virtual functions must be members of some class.
- Virtual functions cannot be static members.
- They are accessed through object pointers.
- A virtual function must be defined in the base class, even though it is not used.
- The prototypes of a virtual function of the base class and all the derived classes must be identical(same). If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
- We cannot have a virtual constructor, but we can have a virtual destructor

Consider the situation when we don't use the virtual keyword.

```
#include<iostream>
using namespace std;
class Base
{
    public:
    void display()
    {
        cout << "Base Display Function
"<<endl;
    }
};
class Derived: public Base
{
    public:
    void display()
    {
```

```
        cout<<"Derived Display
Function"<<endl;
    }
};
int main()
{
    Base *bptr,B;
    Derived D;
    bptr = &B;

    cout<<"Pointed to Base:\n";
    bptr->display();
    cout<<"Pointed to Derived:\n";
    bptr=&D;
    bptr->display();
    return 0;
}
```

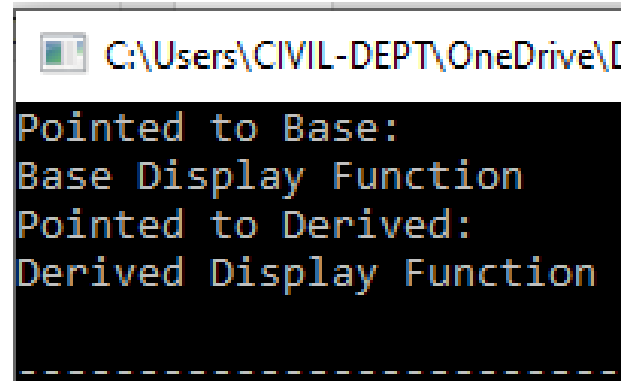


C:\Users\CIVIL-DEPT\OneDri
Pointed to Base:
Base Display Function
Pointed to Derived:
Base Display Function

Using Virtual

```
#include<iostream>
using namespace std;
class Base
{
    public:
    virtual void display()
    {
        cout << "Base Display Function "<<endl;
    }
};
class Derived: public Base
{
    public:
    void display()
    {
        cout<<"Derived Display Function"<<endl;
    }
}
```

```
};
int main()
{
    Base *bptr,B;
    Derived D;
    bptr = &B;
    cout<<"Pointed to Base:\n";
    bptr->display();
    cout<<"Pointed to Derived:\n";
    bptr=&D;
    bptr->display();
    return 0;
}
```



```
C:\Users\CIVIL-DEPT\OneDrive\I
Pointed to Base:
Base Display Function
Pointed to Derived:
Derived Display Function
-----
```

Pure Virtual Function

- A virtual function is not used for performing any task. It only serves as a placeholder.
- When the function has no definition, such function is known as "do-nothing" function.
- The "do-nothing" function is known as a pure virtual function. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
- A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.
- The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.
- Pure virtual function can be defined as:

`virtual void display() = 0;`

Example

```
#include <iostream>
using namespace std;
class Base
{
    public:
    virtual void show() = 0;
};
class Derived : public Base
{
    public:
    void show()
    {
        cout << "Derived Class Function"<< endl;
    }
};
int main()
{
    Base *bptr;
    //Base b;
    Derived d;
    bptr = &d;
    bptr->show();
    return 0;
}
```

Abstract Class

- Abstract Class is a class which contains at least one Pure Virtual function in it. Abstract classes are used to provide an Interface for its sub classes. Classes inheriting an Abstract Class must provide definition to the pure virtual function, otherwise they will also become abstract class.
- Characteristics of Abstract Class:
 - Abstract class cannot be instantiated, but pointers and references of Abstract class type can be created.
 - Abstract class can have normal functions and variables along with a pure virtual function.
 - Abstract classes are mainly used for act as a base, so that its derived classes can use its interface.
 - Classes inheriting an Abstract Class must implement all pure virtual functions, or else they will become Abstract too.

Example

```
class Base
{
    public:
        virtual void show() = 0;    // Pure Virtual Function
};

class Derived:public Base
{
    public:
        void show()
        {
            cout << "Implementation of Virtual Function in Derived class\n";
        }
};

int main()
{
    Base obj;    //Compile Time Error
    Base *b;
    Derived d;
    b = &d;
    b->show();
}
```